

Sensoren anschliessen und pruefen

Anleitung fuer den Fall, dass ein Parkfeld **belegt** ist, die App es aber **nicht anzeigt**. Sie fuehrt in wenigen Minuten zur Ursache, ohne dass jemand Code aendern muss.

Werkzeug dafuer ist die Diagnose-Seite:

```
http://<IP-des-Pi>:5000/diag
```

Zuerst: Sehe ich ueberhaupt echte Sensoren?

Bevor irgendetwas gemessen wird, muss zweifelsfrei feststehen, dass die angezeigte Seite **nicht** die Simulation ist. Drei Merkmale, alle sofort sichtbar:

Merkmal	Echte Sensoren (gpio)	Simulation
Abzeichen oben rechts	gruenes LIVE	rotes SIMULATION
Banner oben auf der Seite	keines	grosser roter Kasten „Simulationsmodus – keine echten Sensoren“
Tippen auf eine Kachel	passiert nichts	Kachel schaltet um

Zusaetzlich nennt das Banner den **Rechnernamen**, mit dem der Browser verbunden ist. Steht dort der Name eures Laptops statt des Pi, ist die falsche Adresse geoeffnet.

Hart nachpruefen laesst es sich hier:

```
curl http://<IP-des-Pi>:5000/api/health
```

```
{ "status": "ok", "mode": "gpio", "live": true,  
  "host": "raspberrypi", "sensors_ok": 7, "sensors_total": 7 }
```

- `"live": true` und `"mode": "gpio"` → echte Sensoren.
- `"live": false` → **Simulation**, es wird nichts gemessen.
- `"status": "degraded"` → einzelne Sensoren fehlen, siehe `sensors_failed`.
- `"status": "error"` → **kein einziger** Sensor liefert Daten.

Damit das gar nicht erst passieren kann: ANNA_STRICT

Ist `ANNA_STRICT=1` gesetzt (im mitgelieferten systemd-Dienst standardmaessig), **startet das Backend gar nicht**, wenn der gpio-Modus verlangt ist, aber kein einziger Sensor geoeffnet werden konnte. Ein Dienst, der sichtbar nicht startet, ist deutlich besser als eine Web-App, die ueberzeugend aussieht und in Wirklichkeit nichts misst.

```
sudo systemctl status anna      # zeigt den Startfehler im Klartext  
journalctl -u anna -b --no-pager | tail -20
```

Das Prinzip: Rohpegel statt Auswertung

Die normale App zeigt nur das Ergebnis („frei“ / „belegt“). Fuer die Fehlersuche ist das zu wenig, denn zwei voellig verschiedene Fehler sehen dort gleich aus. Die Diagnose-Seite zeigt deshalb zusaetzlich:

Spalte	Bedeutung
Rohpegel	Was elektrisch am Pin anliegt: HIGH (3,3 V) oder LOW (GND)
Auswertung	Was die Software daraus macht (beruecksichtigt pull_up und invert)
Wechsel	Wie oft sich der Pegel geaendert hat, seit das Backend laeuft

Diagnose zeigt ungefiltert, die App geglaettet. Die Park-App entprellt die Sensorwerte (settings.confirmations, Standard 2): Ein Zustand wird erst uebernommen, wenn er zweimal hintereinander gemessen wurde. Die Diagnose-Seite umgeht das bewusst und zeigt den Sensor so, wie er wirklich ist. Weichen beide fuer ein bis zwei Sekunden voneinander ab, ist das **kein Fehler**, sondern genau diese Glaettung.

Die Spalte **Wechsel** ist der Schluessel. Stelle ein Auto auf ein Feld und nimm es wieder weg:

- **Wechsel bleibt 0** → Das Signal kommt **gar nicht am Pi an**. Der Fehler liegt in Verdrahtung, Pin-Nummer oder Sensortyp. Die Software ist unschuldig.
- **Wechsel zaehlt hoch, aber „Belegt“/„Frei“ ist vertauscht** → Das Signal kommt an, nur die **Auswertung** stimmt nicht. Ein Klick auf invert genuegt.

Schritt 1 – Laeuft ueberhaupt der GPIO-Modus?

Der Echtbetrieb ist der **Standard**: python run.py liest immer die echten Sensoren. Eine Simulation entsteht nur, wenn jemand sie ausdruecklich anfordert (ANNA_BACKEND=simulated). Trotzdem zuerst pruefen:

```
curl http://localhost:5000/api/health
```

- "live": true und "mode":"gpio" → richtig, echte Sensoren.
- "live": false → jemand hat den Simulator gesetzt, es wird nichts gemessen.

Wo die Einstellung herkommen kann:

```
systemctl show anna -p Environment      # Dienst-Umgebung pruefen
env | grep ANNA_                        # Umgebung der eigenen Shell
```

ANNA_BACKEND=simulated entfernen, dann sudo systemctl restart anna.

Schritt 2 – Melden sich alle Sensoren?

Auf `/diag` muss in der Spalte ganz rechts jedes Feld `ok` sein. Steht dort ein Fehler wie `GPIO17 is already in use`, konnte der Pin nicht geöffnet werden. Mögliche Gründe: ein zweiter ANNA-Prozess läuft noch (`sudo systemctl stop anna` vor dem manuellen Start!), oder der Pin ist von einer anderen Funktion belegt.

Schritt 3 – Bewegt sich der Pegel?

Auto auf ein Feld stellen, wieder wegnehmen, dabei die Spalte **Wechsel** beobachten (die Seite aktualisiert sich selbst).

- Zählt hoch → weiter bei Schritt 5.
- Bleibt 0 → weiter bei Schritt 4.

Schritt 4 – Den richtigen Pin finden

Auf `/diag` „**Pin-Suche starten**“ drücken und **während der 6 Sekunden** das Auto auf das gesuchte Feld stellen oder wegnehmen. Das Backend beobachtet dabei alle brauchbaren GPIO-Pins und meldet, welcher sich bewegt hat.

Ohne Browser geht dasselbe am Pi:

```
python scripts/gpio_check.py --scan
```

Ergebnis „ein Pin hat gewechselt“: Das ist der tatsächlich verdrahtete Pin. Über „zuweisen“ direkt übernehmen (Backend mit `ANNA_DIAG=1` gestartet) oder in `config/parking_layout.json` eintragen.

Ergebnis „kein einziger Pin hat sich bewegt“: Dann liegt es an der Hardware:

1. **Gemeinsame Masse?** Sensor-GND und Pi-GND müssen verbunden sein. Ohne gemeinsames Bezugspotenzial misst der Pi nichts Sinnvolles.
2. **Reagiert der Sensor überhaupt?** Ein **Reed-Kontakt** schaltet nur bei einem **Magnetfeld** – ein Modellauto aus Stahl ist *kein* Magnet. Es braucht einen kleinen Magneten unter dem Auto (siehe `Architekturkonzept.md`, Abschnitt 4.1). Zum Test: Magnet direkt an den Reed halten – wechselt der Pegel jetzt?
3. **Sensor mit Elektronik statt Kontakt?** Dann muss die Beschaltung passen, siehe unten „Sensortypen“.

Schritt 5 – Auswertung korrigieren

Wechselt der Pegel, aber „Belegt“/„Frei“ ist vertauscht: auf `/diag` beim Feld `invert` an klicken. Das wird direkt in `config/parking_layout.json` gespeichert und sofort übernommen – kein Neustart nötig.

Die häufigste Falle: BCM oder Header-Pin?

Dieselbe Zahl bedeutet zwei verschiedene Kontakte:

- **BCM** ist die GPIO-Nummer (GPIO17). So versteht `gpiozero` alle Zahlen.
- **Board** ist die durchnummerierte Position auf der 40-poligen Steckerleiste.

Fuer unser Layout ergibt das:

Feld	Zahl	Als BCM (Software liest)	Als Header-Pin (evtl. verdrahtet)
B1	17	GPIO17 (Header 11)	3V3-Versorgung – nie ein Signal
B2	27	GPIO27 (Header 13)	ID_SD / EEPROM – reserviert
B3	22	GPIO22 (Header 15)	GPIO25
B4	23	GPIO23 (Header 16)	GPIO11 (SPI SCLK)
H1	24	GPIO24 (Header 18)	GPIO8 (SPI CE0)
H2	25	GPIO25 (Header 22)	GND – nie ein Signal
H3	5	GPIO5 (Header 29)	GPIO3 (I2C, feste Pull-ups)

Wurde nach **Header-Nummern** verdrahtet, koennen B1 und H2 grundsatzlich nie etwas melden, und die uebrigen Felder liegen auf falschen Pins.

Eindeutiges Erkennungsmerkmal. Rechnet man beide Listen gegeneinander, gibt es genau eine Ueberschneidung: Header-Pin 22 ist GPIO25 – und GPIO25 ist im Layout das Feld **H2**. Wenn also ein Auto auf **B3** dazu fuehrt, dass in der App **H2** als belegt erscheint, ist die Verwechslung damit bewiesen.

Vorsicht beim Testen. Haengt ein Reed-Kontakt tatsaechlich an Header-Pin 17 (3V3) und schaltet gegen GND, wird beim Auflegen des Autos die 3,3-V-Versorgung kurzgeschlossen – der Pi kann abstuerzen oder neu starten. Wer das vermutet, misst **vorher stromlos** mit dem Multimeter durch, statt es auszuprobieren.

Loesung ohne Umverdrahten – in `config/parking_layout.json`:

```
"settings": { "numbering": "board" }
```

Danach sind alle `gpio_pin`-Werte als physische Header-Pins zu lesen; die Software rechnet selbst um. Vollstaendige Tabelle:

```
python scripts/gpio_check.py --pinout
```

Sensortypen und Beschaltung

Einstellbar pro Feld in `config/parking_layout.json`:

Sensor	Verhalten bei Belegung	Einstellung
Reed-/Magnetschalter gegen GND	zieht den Pin auf LOW	<code>"pull_up": true</code> (Standard)

Induktiver Sensor NPN (Open Collector)	zieht auf LOW	<code>"pull_up": true</code>
Induktiver Sensor PNP	legt aktiv HIGH an	<code>"pull_up": false</code>
Modul-Platine mit eigenem Ausgang	treibt HIGH und LOW	<code>"pull_up": null + "active_state": true/false</code>

Spannung beachten: Die GPIO-Eingaenge des Pi vertragen **maximal 3,3 V**. Industrielle Naeherungsschalter arbeiten oft mit 12–24 V und geben diese am Ausgang aus – ohne Pegelwandler oder Spannungsteiler zerstoert das den Pi.

Behobener Softwarefehler (Stand dieser Version)

Bis einschliesslich Commit `bbcb1d4` erzeugte `app/main.py` beim blossen **Importieren** bereits eine komplette Anwendung (`app = create_app()` auf Modulebene). Da `run.py` das Modul importiert *und* danach regulaer eine App erzeugt, wurden die GPIO-Pins **zweimal** geoeffnet. Der zweite Versuch scheiterte mit „GPIO.. is already in use“ – und zwar fuer **jeden** Pin.

Weil das Backend einzelne Pin-Fehler bewusst abfaengt (damit ein defekter Sensor nicht den ganzen Dienst lahmlegt), passierte das **ohne sichtbare Fehlermeldung**: `read_all()` lieferte gar keine Werte, alle Felder blieben dauerhaft „frei“.

Das ist behoben (kein App-Objekt mehr auf Modulebene) und durch zwei Tests abgesichert. Falls ihr auf einem aelteren Stand testet, ist das die erste Ursache, die ihr ausschliessen solltet – erkennbar an:

```
journalctl -u anna -f | grep "nicht nutzbar"
```